in process of being translating, put the "translation-in-progress" label on it. This enables the team to easily find all blog posts in Contentful that are in progress of being translated and reviewed.
- Remove and apply local label to languages the post is translated into.
- Here's a direct link to the Contentful Blog space to see all posts currently being translated.
- If you make changes to translated content in the Contentful blog space, note your changes in this spreadsheet. (need link to sheet)

Tag	Definition	How to use
translation-in-progress	Notes when a blog post is currently be translated and reviewed	Apply when translation request is opened. Remove when post is published
languagede-DE	Marks entry that is in German	Apply to blog post that is translated into German
languagefr-FR	Marks entry that is in French	Apply to blog post that is translated into French
languageja-JP	Marks entry that is in Japanese	Apply to blog post that is translated into Japanese

---
SECTION: marketing/blog/blog-metrics.md

You can find the GitLab Blog Dashboard here.

You can learn more about our other website analytics dashboards here.

SECTION: marketing/blog/git-guide.md

General Tips

Please note that this guide assumes some knowledge of how to use GitLab, and how to use Git on your local machine. If you're unsure of what something means, you may need to go and review GitLab 101 or Editing the handbook, which explains how to set up your computer to edit the website locally.

Make sure your Terminal is up to date

When you sign on for the day, try to do a git pull origin master in the terminal so that you have the most recent files and changes from others locally on your computer · this will avoid merge conflicts and overwriting files.
When you do this, make sure you are in the right directory, (e.g. MacBook-Pro:www-gitlab-com user$).

Throughout the day you can also run git pull to get the most recent changes locally.

Committing Changes and Pushing Changes

Committing changes is like saving to your computer. You edit a blog post file, or add a new one, then commit it to save it. Pushing changes is like uploading those changes to a shared directory (such as Google Drive). This means that other team members will be able to see your changes in your MR on GitLab.com.

To start, make sure you're on the correct feature branch with git checkout 0000-branch-name[enter] in terminal.

Add, change, update files in the repository

In Terminal · run git status [enter] to see all of the files you've modified. Git add . [enter] will stage all of these files (and any you created) for a commit.

Next we need to add a commit message with git commit -m "[descriptive message goes here]" [enter].

Now we're ready to push your local changes to 0000-branch-name using git push origin 0000-branch-name [enter].

Commit Early and Often

Commit early and often. Avoid working on multiple large files and then committing and pushing the files all at once (it can take several minutes, or even hours, to push depending on your internet connection · worst case it times out).
Run git pull [enter] on occasion to make sure you have the most recent changes / updates locally.

Static Site Editor Developer Tips

There is a handy resource the Static Site Editor group has put together for Git Tips here.

Getting Recent Changes From Main

How and When to Use Merge Workflow

Use a merge workflow for feature branches where MULTIPLE PEOPLE ARE WORKING ON THEM. Use the built-in squash and merge functionality when merging an MR to ensure only clean, atomic, squashed commits make it to master.

```
console
git checkout 0000-branch-name
git fetch
git merge origin/master
git push origin  0000-branch-name
```

What is Rebasing and Why Should I Care?

If you are working on a merge request for some time and have committed a lot of changes, you may need to rebase (i.e. update) your MR to reflect any changes that other people may have made

to the main branch of the website. This helps to prevent merge conflicts (where someone else's change conflicts with your change, and Git doesn't know which change to accept) or pipeline failures because of technical changes that aren't reflected in your outdated MR.

To rebase, run these commands in the terminal:

```
console
git checkout 0000-branch-name
git fetch
git rebase origin/master
git push origin  0000-branch-name
```

How to Resolve Merge Conflicts

Official GitLab Documentation

Here is the official documentation on merge request conflict resolution in GitLab.

Here's a great blog post on resolving merge conflicts from the GitLab UI.

Merge conflicts if you haven't made any changes to your local branch

If you have not made any changes to your local branch and are getting a conflict message from origin, just reset your local branch to be exactly like origin. WARNING: If you have made changes to your blog post, those changes will be lost!

```
console
git fetch
git checkout 0000-branch-name
git reset -hard  origin/0000-branch-name
```

Help with Git

Slack Channels

If you're having problems with Git, reach out in the following Slack channels =)

- git-help
- mr-buddies

Existing Resources

Here are some existing resources from GitLab for Git.

- Developer Cheatsheet, Engineering Handbook
- Git Cheat Sheet for Press

SECTION: marketing/blog/release-posts/_index.md

Introduction

Release posts are blog posts that announce changes to the GitLab application. This includes our regular cadence of monthly releases which happen every month, and patch/security releases whenever necessary.

Release posts follow a process outlined here, and the templates that are used to create them also highlight what needs to be done, by whom, and when those items are due.

Quick Links

- Frequently used templates
- Helpful reference pages
- Release post scheduling

Schedule

At a high level, the Release post schedule is:

Monday, 3 weeks before release

- Release Post Manager manually trigger the following scheduled pipelines in the www-gitlab-com project:
  - Release Post Process Kickoff Tasks
- These invoke the bin/rake releasepost:start rake task. (pipeline configuration; rake task)
- This task creates the branches, MRs, and issues necessary to run the Release Post process
- The MRs and issues will be assigned to the Release Post Manager using the content in releasepostmanagers.yml
- After the Release Post Process Kickoff Tasks pipeline completes, and the release post branch is created with associated merge request, manually trigger the following scheduled pipelines in the www-gitlab-com project:
  - Add deprecations and removals to current release post branch

Monday through Thursday, 3 weeks before release

- PMs contribute MRs for their content blocks
  - Features and Upgrades are contributed as release post item MRs targeting the release post branch
  - Primary items are added to features.yml
  - Recurring content blocks for Omnibus, GitLab Runner, and Mattermost are added by the area owner
  - Non-standard product announcements, uncategorized items, and other announcements can be announced using the extras content type
- EMs and PMs announce deprecations and removals

Thursday, 1 week before release

Code cutoff

- EMs and PMs make sure items that are feature flagged are enabled by default to ensure inclusion into the self-managed release.
- Deprecation and removal MRs are assigned to TWs for final review and merge.
- TW Reviewers finish review of Features, Deprecations, Removals, Upgrades, and Extras
- PMMs, Product Design Managers, Product Designers, and PM Leaders do optional reviews of release post item MRs
- EMs:
  - Merge feature release post item MRs if the underlying code was merged by the Thursday, 1 week before release
  - Merge feature release post item MRs if manually verified to be in the release
    - MRs can be manually verified using the /chatops run release check <MR URL> <RELEASE> chatops command
- TW Reviewers merge deprecation and removal MRs

{{% note %}}
MRs added after the Thursday, 1 week before release should target the release-x-y branch, not master
{{% /note %}}

Monday of release week

- At <time datetime="16:00">4 pm UTC (11 am ET / 8 am PT)</time>, another release post automation task (scheduled pipeline; rake task) performs content assembly
- Release Post Manager picks features to highlight and creates the introduction content

Monday through Tuesday of release week

- Contributor Success adds the Notable Contributor
- Release Post Manager and Technical Writer perform final reviews
  - Changes after <time datetime="16:00">4 pm UTC (11 am ET / 8 am PT)</time> on the Monday of release week will be done via the release-X-Y branch and are subject to approval by the Release Post Manager.
  - The TW Lead verifies the deprecations and removals links in the release post
  - RPM create a What's New MR

{{% note %}}
The Monday through Tuesday of release week can fall on vacations or holidays. PMs should designate who to respond to time-sensitive inquiries should they be unreachable. Release Post Managers are empowered to make decisions and display bias for action if they haven't received a response by EOD on the Tuesday of release week.
{{% /note %}}

Thursday, release day

- Release team publishes the latest package
- After the package is released, the Release Post Manager publishes the release post to the master branch
- The GitLab.org Releases page will also populate the changelog via an automated process when

release posts are published (pipeline task)

{{% note %}}
Details for all of these steps are described in the Monthly release post MR template and the Monthly release post item MR template.
{{% /note %}}

Participants

- Release Post Manager
- PM contributors
- PMM reviewers
- PMM lead
- TW lead
- Product Design reviewers
- TW reviewers
- Technical advisors
- Engineering Managers

Volunteering for the Release post

Each month, a Product Manager, Technical Writer, and an Engineering Department Technical Advisor volunteer to manage the release post, as listed in the Release Post Scheduling page. Product Marketing Managers also sign up, but mostly as shadows for awareness for their related marketing activities. The Product Manager volunteer will lead the release post as the Release Post Manager and is listed as the Author of the release post when the post is published. To update the release post scheduling list, all volunteers need to edit the data file below:

- Data YAML file: gathers the Release Post Managers for every release (9.0 onwards). Be sure to update the "Managers" section below the "Versions" if this is your first release.

It's highly recommended that all volunteers shadow the release post prior to the one they run. Volunteers can update the previously mentioned data YAML file to indicate both when they shadow and when they help run the release post.

Release Post Managers will need Maintainer access privileges for the https://gitlab.com/gitlab-com/www-gitlab-com/ project. If you need access, model your request after this confidential issue.

Release Post Manager

Product Managers of any level (IC or managers) can volunteer for any release that doesn't have someone assigned yet. While we encourage IC product managers to take advantage of this opportunity to demonstrate their leadership skills, we also value that managers will bring their experience to the role.

Before committing to the date of your choice, please be sure you can perform the critical path Release Post Manager tasks between Thursday, 1 week before release, and the release date of the month as defined in the monthly MR template. If you cannot perform any of the Release Post Manager tasks between Thursday, 1 week before release, and the release date of the month,

please sign up for a release post that better aligns with your availability.

To assign yourself as Release Post Manager or Release Post Manager's shadow, simply add your name on the Release Post Scheduling page by submitting an MR to update the /data/releasepostmanagers.yml file. Otherwise, PMs will be assigned using a fair scheduling principle leveraging this tracking doc:

1. Members that never managed a release post before
1. Members that have the longest interval since they managed their last release post

After joining the company, there is a grace period of a few months where the new Product Manager
will get up to speed with the process, then they will be scheduled to manage a release post.

Adding members to the list is a shared task, and everyone can contribute by following the principle described above. Scheduled people are pinged in the merge request to make them aware. They don't need to confirm or approve, since they can always update the list if they are not available for the given release post.

{{% alert title="·· Important" color="warning" %}}
If you're scheduled for a given month and you can't make it because you're on vacation, overloaded, or for any other reason, that is okay, as long as you swap the Release Post Manager role with someone else before creating the merge request and starting the whole process. If you take it, you're responsible for the entire process and must be available to carry it out until the end.
{{% /alert %}}

Release Post Manager Shadow

Each month, a Product Manager also acts as a shadow to support the Release Post Manager tasks if needed, act as back up on decisions in absence of the Release Post Manager and prepare to run the next release post. By shadowing the month prior to leading the effort, Product Managers are prepared and aware of any shifts in processes or optimizations needed since the last time they participated.

Shadows should remain engaged with the release process by:

- Following the activity in the slack channels
- Attending the weekly standups
- Assisting the Release Post Manager with content reviews and any other tasks they ask for help on

In order to properly onboard the shadow, the Release Post Manager should:

- Set up an initial coffee chat with your shadow the week after the previous release ships to get to know each other and clarify any initial questions from the shadow
- Point the shadow to this page
- Include the shadow in the initial release post MR creation
- Include the shadow on all meetings and as much as possible on activities like reviews or other opportunities where you can work synchronously together

Remember: The goal of the shadow is to get them engaged and aware of the process so they can run one on their own. Include the shadow as much as possible so they can learn and be prepared!

Technical Advisor considerations

We recommend that technical advisors volunteer for at least 2 or 3 release posts in a row to allow proper time for orientation with the process and the ongoing technical backlog.

Technical advisors are expected to:

- Solve problems with Git branch conflicts and Ruby installations.
- Be able to technically contribute to www-gitlab-com source code.
- Resolve some of the backlog issues.

The responsibilities of a technical advisor can be seen in more detail in Technical advisors.

Release Post Manager Responsibilities

Critical path tasks

- Completing all the tasks assigned to the Release Post Manager in the Release Post MR template
  - Reminder: If you cannot perform any of the Release Post Manager tasks between Thursday, 1 week before release, and the release date of the month as defined in the monthly MR template, it is recommended you sign up for another release post. In the case that schedule/circumstances changes after you'd already signed up for the release post, please start a thread in product in slack and tag @name of PLT member who is reviewing this month]. The name of the PLT member who is reviewing this month can be found on the [release post scheduling page
- Identify the top feature to highlight on the release post page and collecting feedback from the VP of Product
- Creating the What's New MR and working with the VP of Product to identify what to include in What's New
- Sending out reminders about upcoming due dates
- Merging the release post MR on the release date and ensuring the release post page goes live
- Collecting feedback in the release post retrospective issue during the release post not just for your own challenges, but other team members challenges as they pop on Slack and other places
  - Doing a sync retro with the Technical Writer, the Technical Advisor and the Release Post Manager Shadow sometime between the day after the release date and one week after the release date, to identify and collaboratively complete actions to improve the process and update the handbook/MRs
  - Making sure all the action of the retrospective issue are completed and the issue closed before the next release post automation task runs on the Monday, 3 weeks before relea

Other key tasks

- Running a weekly sync or async standup with the release post team (sync standup required for major releases)
- Reviewing and supporting overall content quality and accuracy of all content published in the release post
- Including the Release Post Manager Shadow as much as possible on activities so they learn

prior to their rotation
- Adding the cover image that is (jpg, png) is smaller than 300KB
- Monitoring the Slack Release Post channel to help answer questions and troubleshoot hurdles
- Pinging the PMs and others as needed in Slack or MRs to help resolve feedback
- Making sure the release post is ready to merge two days before the release date
- Communicate directly with product managers using product on Slack as needed to field questions that come up from viewers of the release post blog once it is live on the release date
- If you need additional support in engaging with the community, the Developer Advocacy team (dev-advocacy-team on Slack) is available to support on release days
- Making sure the auto sorting of secondary features by title (alpha) and stage generally looks good or is revised if need be Content Reviews
- Working with PMs and others as needed to make sure any external blogs they reference in their content blogs go live before the release post blog gets published on the release date
- Making sure the TW Lead is aware if release post items are added or removed after the Monday of release week
- Informing the social team that the release post has been published and it's time to schedule social media posts
- Supporting on tasks specific to major releases if collaborators reach out

How to get started

Make sure you have Maintainer access to project https://gitlab.com/gitlab-com/www-gitlab-com/. If you need access, model your request after this confidential issue.

An automated task will create the branches, MRs, and issues necessary to run the Release Post process, including making the appropriate assignments and mentions based on the Release Post Manager schedule.

If you have not been assigned to a Release Post X.Y MR by the end of the day on the Monday, 3 weeks before relea:

- Work with your Technical Advisor to run bundle exec rake releasepost:start to kickoff the X-Y Release Post, or
- Follow these steps to manually create the release post branch and required directories/files

Communication

The Release Post Manager, the Technical Advisor, the Technical Writer, and PMM Lead will need to communicate about topics that are related to the release post but not relevant to the broader team, these chats should occur in Slack X-Y-release-post-prep channel in Slack, to minimize distractions and unnecessary notifications for the broader team in Slack release-post.

The Release Post Manager posts in Slack channels most frequently with reminders. As such, if the Release Post Manager is seeking guidance on how to phrase certain posts, it's recommended to scroll to the approximate date that post would have been made by the previous Release Post Manager in the relevant Slack channel. However, here are some best practices and an example:

- Make a clear, descriptive statement of what's being shared and why
- If you need someone to take an action, say so explicitly and tag that person

- If the action requested is time sensitive, give a clear due date
- If there are known issues they need to be aware of, list them out
- Always cc your release post team for big announcements so everyone is in the loop

When communicating in either Slack release-post or X-Y-release-post-prep, organize your announcements and requests via unique discussions threads to make it easier to track conversations. For example, avoid combining various reminders just because they fall on the same date when they address different topics. As a general rule, if there's is a unique task list item for the reminder in the MR template, that reminder should get its own separate post whether it is in Slack or the MR itself. Also, review GitLab's effective slack communication guidance.

Sample post to executive stakeholders for review is below.
The name of the PLT member who is reviewing this month can be found on the release post scheduling page

md
@[name of PLT member who is reviewing this month] The 13.6 Release Post has been generated and can be reviewed at https://release-13-6.about.gitlab-review.app/releases/2020/11/22/gitlab-13-6-released/index.html

Please share your feedback by <time datetime="18:00">6 pm UTC (1 pm ET / 10 am PT)</time> on Friday November 20 (tomorrow). Thank you for your review!

Currently there are no known issues/adjustments to the content but I know of one deprecation that needs to be added and will happen with my first wave of edits.

Here's the 13.6 release post MR: https://gitlab.com/gitlab-com/www-gitlab-com/-/mergerequests/66652

cc @TW Lead @tech-advisor @PMM

Other samples for posts include reminders and notices on any items that the Release Post Manager is taking:

md
· Hi team! Announcing a "last call" that no further contributions to the bugs, performance improvements, and usability improvements MRs will be taken after the Thursday, 1 week before release. Please get them in ⋯⋯ cc @[name of PLT member who is reviewing this month]

md
Hey team, reminder that there are currently XX Open and Ready MRs targeting XX.X milestone (link to open MRs). Please take a moment to ask your EMs to merge or to move out the items that won't make milestone.

md
Hi all, I will be completing the final merge for the release post in the next 45 minutes-1

hour! I will be coordinating any activities with team members to resolve any problems that come up. cc @Tech Advisor @TW Lead

The Developer Advocacy Team will reach out to the Release Post Manager in Slack release-post following their Release days process when they need help responding to inquiries about content in the release post blog. These needs will primarily arise within the first week of going live with the blog. However, as the Author for a specific release post, you may get pinged to help coordinate a response some weeks later as issues arise. You will usually just need to find the best DRI to handle the issue, often the PM of the release post item in question.

Sometimes, external PR and Marketing firms reporting on the release or managing media relations may ping the RPM directly with questions, since the RPM is the "author" of the release post. If this happens, the Release Post Manager should figure out who in Marketing can take over this communication.

Content reviews

The due dates for various reviews across all participants can be found on the release post MR template and the release post item MR template. PM contributors are encouraged to cease attempts to add new content blocks after the content merge deadline on the Thursday, 1 week before release, and especially after final content assembly happens on the Monday of release week at <time datetime="15:00">3 pm UTC (11 am ET / 8 am PT)</time>. Exceptions can be made for highly impactful features, but it is up to the discretion of the Release Post Manager to work with the PM to add more content blocks up until the Wednesday, day before release.

Keeping an eye on the various content reviews (TW, PMM, and Director) for the individual release post items (content block MRs) is the responsibility of PM contributor. However, it is recommended that the Release Post Manager keep an eye on how many items are not yet marked with the Ready label on the Thursday, 3 weeks before release of the month or not yet merged on the Thursday, 1 week before release, and check in with PMs in Slack Release Post channel to support and clear hurdles if needed. A really easy way to do this is to keep your eyes on the Preview page and copy-edit and link check items as new items appear. It's also important to do this because this page is LIVE to users and should be error free.

The review and any needed adjustment to the ordering of secondary features due to stakeholder feedback is the responsibility of the Release Post Manager. Secondary features, removals, and upgrade notes are all sorted alphabetically by title, grouped by stage. To affect the sort order of the secondary features, a change to the content block's title is required. The Release Post Manager should work with the product managers of the content blocks to make these changes, to ensure accuracy and alignment.

After the Review App for the release post has been generated, the Release Post Manager solicits additional feedback from the product leaders via Slack in the release-post channel.

It is the Release Post Manager's responsibility to make sure all content is completed by the Tuesday of release week, ensuring a one day buffer is left for final error fixes and small improvements.

NOTE: To the extent possible, we strive to use GitLab's Community Code Review Guidelines when

performing Release Post content review.

What RPM should look for when reviewing content blocks

It is recommended for the Release Post Manager to review all content for quality, including the marketing intro. But when reviewing content blocks in each release post item MRs, the RPM should look for the following:

1. Are the why (problem) and the what (solution) clearly stated? See writing about features as a guideline for what feature descriptions should contain.
2. Do the filenames follow the recommended file-naming convention? See important note on naming files under Instructions for PM contributors.

Tips for reviews

1. Utilize the Available now on GitLab page to easily scan release post items that have been merged.
1. Search the Available now on GitLab and preview pages for characters like [,], (, and) to find malformed links.
1. Copy/paste the content of those pages into a tool like Grammarly to find less obvious typos like duplicate words.

Release post intro content

The introduction content of the release post (found in YYYY-MM-DD-gitlab-X-Y-released.html.md) is templated to be standard across all release posts, and should not be modified without approval from @justinfarris. This file is linked at the top of the release post MR for reference and ease of editing. The Release Post Manager will make sure all primary items are approved and a top feature is designated and ask the VP of Product for feedback.

PM Contributors

Product Managers are responsible for raising MRs for their content blocks and ensuring they are reviewed by necessary contributors by the due date. These are mostly added by the Product Managers, each filling up the sections they are accountable for, but anyone can contribute, including community contributors. Content blocks should also be added for any epics or notable community contributions that were delivered.

Contribution instructions

In parallel with feature development, a merge request should be prepared by the PM with the required content. Do not wait for the feature to be merged before drafting the release post item, it is recommended PMs write Release Post Item MRs as they prepare for the milestone Kickoff.

{{% alert title="Important" color="info" %}}
The Instructions below apply up to the Monday of release week <time datetime="07:59">7:59 am UTC (2:59 am ET / Thursday, 1 week before release 11:59 pm PT)</time>. After content assembly on the Monday of release week, anyone who wants to include a change in the upcoming release post must coordinate with the Release Post Manager and follow detailed instructions in the

Merging content blocks after the Monday of release week section for special handling of late additions.
{{% /alert %}}

Key dates

- During kickoff preparation, or when planning for the upcoming milestone: consider creating the release posts early to enable the team to work backwards
- Thursday, three weeks before the release - Drafted: ready for review by Product Marketing, Tech Writer, and PM Group Manager or PM Director
- Monday through Thursday the week before the release - Reviewed: reviewed by all required stakeholders, content revised as needed and ready to be merged
- Thursday, 1 week before release - Merged: release post item MR merged by the Engineering Manager if feature has been merged
- Monday of release week - Final content assembly: and release post blog content lock in preparation for final reviews/editing

{{% alert title="Important" color="info" %}}
If a feature being announced involves references to external business partners, you'll need to start MR draft approvals earlier. One such example would be Cloud Seed. These types of announcements require extra reviews with GitLab leadership, business partners and Legal team. In these cases, please reach out to @justinfarris to start MR reviews at least one milestone ahead of the milestone in which you want to make the release post announcement.
{{% /alert %}}

Release Post Item Instructions

Option 1: automated MR creation

The release post item generator automates the creation of release post items using issues and epics. Draft your release post content under the Release notes section of the feature issue template and then follow the release post item generator instructions.

{{% note %}}
The generator will not create an MR for a confidential issue. To add a release post item for work relating to a confidential issue, follow the steps below to create an MR manually and remove any confidential information or links.
{{% /note %}}

Option 2: manual MR creation

- Create a new branch from master of the www-gitlab-com repository for each feature (primary, secondary, removal). Deprecations are handled differently
- Open a merge request targeted at the master branch
- Use the Release Post Item template
- Content should be one YAML file added to data/releaseposts/unreleased/ on the master branch
  - See data/releaseposts/unreleased/samples/ for format and sample content
  - Note that the structure needs to be preserved, like features: then primary:, then the feature content
  - Images should be placed in /source/images/unreleased/

- Update the data/features.yml (if applicable) to include your feature and commit the changes as part of the same merge request
- Complete the PM checklist included in the Release Post Item MR template, which includes but not limited to these tasks:
  - Assign the MR to the relevant Tech Writer for review
  - Assign the MR to the relevant Product Marketing Manager, and/or Director if additional review is needed
  - Once all content is reviewed and complete, add the Ready label and assign MR to the appropriate Engineering Manager (EM) to merge when the feature is deployed and enabled.

Important note on naming files: PMs should create file names that are descriptive and have reasonable overlap with the title of the content block itself. This makes it easier to related content blocks to yml file by different participants in the review process. Either underscores or hyphens - can be used as long as the correct prefix is used (stagename, removal, or upgrade) as listed below.

- Feature file names: stagename-featurename.yml (for example, create-group-wikis.yml). Do not:
  - Designate primary vs. secondary as that can change.
  - Use category or group name.
  - Include the reporter's name.

- Removal file names: removal-something-else-descriptive.yml
- Upgrade file names: upgrade-another-description.yml

Some troubleshooting hints:

- Use git merge, don't use git rebase. Rebase is a powerful tool that makes for a clean commit history, but due to the volume of commits by the number of collaborators on the www-gitlab-com repo, it will typically have a lot of conflicts you'll have to manually work through. Since your content MRs should only contain changes relevant to your own content block and a single addition to features.yml, merge conflicts should be minimal, and typically nonexistent. If you start a rebase and run in to issues, you can always back out with git rebase --abort.
- Remember to close your quotes, check your filenames, and indent properly. Many vague pipeline errors are caused by common coding gotchas. Make sure your quotes are closed, the file you're referencing uses exactly the same filename you listed, and you have the right indentation set on each line.

Content

We want to help people understand new features to increase adoption their adoption. In general, release posts should succinctly state the problem to solve, the solution, and how customers benefit from the solution. Be sure to reference your Direction items and Release features. All items which appear
in our Upcoming Releases page should be included in the relevant release post.

When writing your content blocks, be sure to reference Writing about features to ensure your release post item writeups align with how GitLab communicates. For example, we avoid formal phrases such as "we are pleased to announce" and generally speak directly to our users by

saying "you can now do x" rather than "the user can now do x". Checking out the links to these guidelines will help you align our tone/voice as you write, ensuring a smoother and more speedy review process for your release post items.

PM contributors are encouraged to use discretion if wanting to add new content blocks after the final merge deadline of the Thursday, 1 week before release, and especially after final content assembly happens at 8 AM PST (3 PM UTC). But if highly impactful features are released that can not wait till the next blog post, PMs should reach out and coordinate with the Release Post Manager. It is up to the discretion of the Release Post Manager to work with the PM to add more content blocks up until the Wednesday, day before release.

Primary vs. secondary

When creating your content for a Release Post item you'll need to determine if it's a primary or secondary feature. Do this in collaboration with your PMM counterpart and reference this guidance if you're unsure:

A feature should be primary if the feature:

- Matures a category (post release you'd update the category maturity for the category your feature lives within)
- Is new, or a significant improvement - it adds key functionality that did not exist previously or significantly changes existing functionality
- Has high demand from customers or the wider community (measured by discussion or upvotes on an epic/issue)
- Feature ties into a current Marketing narrative or campaign
- All primary features should have a corresponding entry in features.yml as well as a photo or video in the release post item block.
- Beta features may be included as primary, or secondary items, but must clearly reflect the Beta status.
- Experimental features are not to be included as primary or secondary items to the release posts.

Experimental Features

To include an experimental feature in the release post, use the experiment template, when creating a release post item in the unreleased directory. Experiment features are displayed in their own section of the release post.

Reviews

PM Director/Group Manager, PMM, and Product Design reviews are highly recommended, but the Tech Writer review is the only one required for inclusion in the Release Post. Tech Writer review is required even when late additions are made to the release post after the Monday of release week. The Tech Writing review should be focused on looking for typos, grammar errors, and helping with style. PMs are responsible for coordinating any significant content/tech changes. Communicating priority about which release post items are most important for review will help Product Section leads, PMMs, and Tech Writers review the right items by the Thursday, 3 weeks before release, to ensure the proper labels are applied to the MR and assign reviewers to the MR when it is ready for them to review (ex: Tech Writing, Direction, Deliverable, etc).

- Note: For consistency, use the Reviewers for Merge Requests] feature in GitLab when assigning PM Director/Group Manager, PMM, TW, and Product Design team members for content reviews.

Recommendations for optional PM Director/Group Manager and PMM Reviews

As PMM reviews are not required, but recommended - and Product Leader and Product Design reviews are optional - PMs should consider a few things when determining which content blocks to request a review for:

- Does the feature contribute to a Group or Stage's overall Direction?
- Does the feature contribute to increasing a Category's maturity?
- Does the feature increase our ability to compete in the market?
- Does the feature have considerable customer demand?
- Does the feature represent a significant UX improvement?

If the answer to any of these is "yes", it is recommended that you coordinate with your Director, PMM, and Product Design counterpart to review the content block by the Thursday, 1 week before release. As the PM it is your responsibility to communicate what MRs need a review from the TWs, PMMs, Product Designers, and Directors as well as the MRs relative priority if you have multiple content block MRs that need reviews.

Merging Content Block MRs

Engineering Managers are the DRIs for merging these MRs when the feature is merged into the codebase itself. This allows all of the relevant parties (Product Managers, PMMs, Product Designers, Section Leads, Technical Writers) to have enough time to review the content without having to scramble or hold up engineering from releasing a feature.

To enable Engineering Managers to merge their feature blocks as soon as an issue has closed, please ensure all scheduled items you want to include in the release post have content blocks MRs created for them and have the Ready label applied when content contribution and reviews are completed.

Reviewing, editing and updating merged content blocks

After content block MRs are merged, they can be viewed on the Preview page and should be updated/edited via MRs to master up until the final merge deadline of the Thursday, 1 week before release. Starting on the Monday of release week, content block MRs should be viewed in the Review app of the release post branch after final content assembly, and updated/edited on the release post branch by coordinating with the Release Post Manager. From the release date forward you should view the content blocks on the blog. It's important to check this page after the content block MR is merged because this page is LIVE to users and should be error free.

Adding, editing, or removing merged content blocks during release week {adding-editing-removing-before-release-date}

After the content assembly starts on the Monday of release week and before the end of Tuesday of release week, adding any new or removing any merged release post items must be coordinated with the Release Post Manager.

This is necessary to allow them to assess the impact on the release post and coordinate any necessary adjustments with the release post team (Tech Writer, PM, etc.). Failure to do so might result in your changes not being picked into the release post.

Before pinging the Release Post Manager, ask yourself if your content absolutely needs to be part of the current release post. At end-of-day on the Tuesday of release week, no late content blocks will be accepted.

Requesting a late addition during release week {requesting-late-addition-before-release-date}

- Ping the Release Post Manager (RPM) in release-post to request adding a new late addition for the release post, and wait for the RPM to give confirmation to proceed. New late additions are release post items that were created after content assembly has already run. The Release Post Manager will do their best to accommodate the request, but it is not guaranteed.
- If the RPM approves the late addition, then PM and RPM will proceed by:
  - PM edits the release post item MR and updates the target branch to be on the release post release-X-Y branch.
  - PM rebases the release post item MR on top of release-X-Y branch.
  - PM moves the RPI yml file and images from /data/releaseposts/unreleased to /data/releaseposts/xy/.
  - PM moves any images from /source/images/unreleased to /source/images/xy/
  - PM Ensure that the imageurl field in the release post yml file points to the image file under /source/images/xy/.
  - PM requests a review of the release post item MR from the Release Post Manager and release post tech advisor. Quick action: /assignreviewer RP-manager
  - PM notifies release post team in the X-Y-release-post Slack channel that the late addition has been requested with a link to the MR.
  - The MR can be approved and merged by the Release Post Manager.
- If the feature is primary and you had not previously added it to features.yml, you will need to create a second MR, branched from master to add the feature to features.yml. (features.yml should be merged to master, not the release post branch).

Process for removing merged content blocks

- Ping the Release Post Manager in Slack release-post to notify them you need to remove an item already merged onto the release X-Y branch.
- Either the Release Post Manager or the PM, with approval from the Release Post Manager, will remove YAML and image files from the release X-Y branch.
- The PM will remove the feature from features.yml on master.

Adding, editing, or removing merged content blocks after the release date {adding-editing-removing-after-release-date}

You can make changes to the release post after it's live to make edits to feature content blocks.

To edit a content block:

1. At the bottom of the release post you wish to edit, select "Edit this page".
1. Find and edit the relevant .yml file in the correct subdirectory. For example, to add or

edit the example Widgets feature to the 14.6 release post, create or edit the data/releaseposts/146/widgetsexample.yml in an MR against master.

To remove the feature block, remove the file in your MR. Or to announce it in the next release post, move the file to the data/releaseposts/unreleased folder.

1. For review and approval, assign the current cycle's Release Post Manager a Reviewer.

To edit a deprecation, follow Editing a deprecation announcement entry.

Accountability

You are responsible for the content you add to the blog post. Therefore, make sure that:

- All new features in this release are in the release post.
- All the entries are correct, especially with regard to links to the documentation or feature pages (when available).
- Feature tier availability: all contain the correct entry.
- All primary features are accompanied by their images.
- All new and/or primary features are added to data/features.yml with a screenshot accompanying the feature (if the feature is visible in the UI).
  - All images are optimized according to the image guidelines and smaller than 150KB.
  - Keep in mind the features.yml is the SSOT for displaying features across about.gitlab.com.
- All features should have a clear value driver.

As noted in the Release Post Item template:

- Make it clear if a feature is new, or is an improvement to an existing feature.
- Make sure your content is reasonably aligned with guidance in Writing about features.
- Ensure that titles use sentence case with feature and product names in capital case.

Write the description of every feature as you do to regular blog posts. Please write according to the Markdown guide.

{{% alert title="·· Important" color="info" %}}
Make sure to merge master into the release post branch before pushing changes to any existing file to avoid merge conflicts. Do not rebase, do git pull origin master then :wq.
{{% /alert %}}

PMs checklist

Once the PMs have included everything they're accountable for, they should check their item in the release post MR description:

By checking your item, you will make it clear to the Release Post Manager that you have done

your part in time (during the general contributions stage) and you're waiting for review. If you don't check it, it's implicit that you didn't finish your part in time, despite that's the case or not.

Once all content is reviewed and complete, add the Ready label and assign this issue to the Engineering Manager (EM). The EM is responsible for merging as soon as the implementing issue is deployed to GitLab.com, after which this content will appear on the GitLab.com Release page and can be included in the next release post. All release post items must be merged on or before the Thursday, 1 week before release. If a feature is not ready by the Thursday, 1 week before release deadline, the EM should push the release post item to the next milestone.

Notes for PMs

Vacations

If you are on vacation before/during the release, fill all your items and create placeholders in the release post Yaml file for all the items you cannot add for whatever reason. To complete them, and to follow up with all the content you are responsible for, assign someone to take over for you and notify the Release Post Manager.

Replies

Please respond to comments in the MR thread as soon as possible. We have a non-negotiable due date for release posts.

Documentation

Please add the documentationlink at the same time you add a content block to the release post. When you leave it to add it later, you will probably forget it, the reviewer will ping you later on during the review stage, and you will have little time to write, get your MR reviewed, approved, merged, and available in the documentation.

Always link to the "EE" version of GitLab docs https://docs.gitlab.com/ee/ (not /ce/) in the blog post, even if it is a CE feature.

PMM Reviewers

Messaging review

Each PM is responsible for pinging their PMM counterpart when they need a review on the messaging for a Release Post Item MR or changes to features.yml.

- Leave comments for the PMs on the items file in the MR. Make sure to comment in the diff on the line that you are referring to so that the PM has the context and comments can be resolved appropriately.
- See writing about features as a guideline for what feature descriptions show have.
- Review the messaging for these features look for these 5 elements:
  - problem/solution: Does this describe the user pain points (problem) as well as how the new feature removes the paint points (solves the problem)?
  - short/pithy: Is this communicated clearly with the fewest words possible?

  - tone clarify: Is the language and sentence structure clear and grammatically correct? Is the text in the present tense, and is "you" used instead of "user."
  - technical clarity: Does the description of the feature make sense for various audiences, including folks who are not deeply familiar with GitLab?
  - value driver: Does the feature help our users Increase Operational Effectiveness, Deliver Better Products Faster, or Reduce Security and Compliance Risk?
- To understand the feature better look at the issue and MR for the feature, they are linked in the YAML. Sometimes the issue description will include the value prop. Read the comments in the issue and MR for the feature, often users and customers will chime in with why they want a feature and what pain the lack of the feature is causing.
- The release post and features.yml can have the same or very similar content - e.g. same screen shot.
  - The tone of the release post is more about introducing the feature "we're happy to ship XYZ..."
  - The tone of features.yml should be evergreen to appear on our website in various places.

PMM Lead

PMM lead is responsible for creating a release post highlight blurb for consumption by field and PR.

The tasks are included in the release post MR template and in the monthly release post intro document.

On or before the third thursday of the month:

- Create new Product marketing issue with PMM-Release-Post template.
- Create release highlights - 3-4 themes with description. Use this document to document your highlights
- Update the issue with the highlights
- Update highspot
  - Add the actual release post blog as a new piece of content in Highspot (Customer Outreach spot) (e.g., release post)
  - Add this new release post on highspot to the GitLab Releases spot overview page in the GitLab Release Post section
  - Create a new pitch template for this release in Highspot (Company Pitch Templates spot) (e.g., pitch template)
  - Add this new pitch template to the GitLab Releases spot overview page in the Release Pitch Templates section
- Flag to comms to share in sales
- Share with the PR and Field enablement team and tag release post manager.

TW Lead

{{% alert title="Note" color="info" %}}
Technical writers review the individual release post items according
to the stage/group they are assigned to.
Each month, one of the technical writers is also responsible for the structural
check of the final release post merge request. This section is about the latter.
{{% /alert %}}

The TW Lead is responsible for a final review of:

- Release post top feature For any identified issues, inform the TW reviewer to resolve as appropriate.
- Release post primary features For any identified issues, inform the TW reviewer to resolve as appropriate.
- Frontmatter check
- Verifying the deprecations and removals sections in the release post link to GitLab the corresponding pages in GitLab Docs.

While individual TW reviewers and product managers have ultimate responsibility for the style and language of their release post items, including deprecations, removals, breaking changes, and Upgrades, TW leads still have an overall responsibility to notify the Release Post Manager, the product managers and TW reviewers if style and language don't seem reasonably consistent (things are obviously out of sync with known guidelines). But it is not the responsibility of the TW leads to fix style and language inconsistencies. However, TW leads do have the responsibility and ownership to make sure that all links in the release post point to relevant content and be fixed, if issues are found.

Consideration: When communicating with your release post team, use the release post prep channel and organize discussions into threads to make it easier to track conversations. Also, review GitLab's effective slack communication guidance.

Structural check

A technical writer, once assigned to the release post merge request, will check the syntax and the content structure.

The Structural check checklist in the main release post merge request description will guide them through the structural check.

Given that the technical writing review occurs in release post items' merge requests, the purpose of the structural check is:

- Review the overall post for consistency. For example, if there's an entry in a previous release post that deprecates an item called auth-server for this date, raise questions if there's also an entry that removes an item referred to as authserver.
- Make sure the post renders well.
- The content as a whole clearly describes the new features and feature improvements.
- Check all the links work and are in place.
- Check all content for syntax errors, typos and grammar mistakes, remove extra whitespace.
- Verify that the images look harmonic when scrolling through the page (for example, suppose that most of the images were screenshots taken of a large portion of the screen and one of them is super zoomed. This one should be ideally replaced with another that looks more like the rest of the images).
- This should happen in the release post item review, but if there's time, double-check documentation links and product tiers.
- Make sure the current release's deprecations and removals also show up in the deprecations doc.

Pay special attention to the release post Markdown file, which adds the introduction.
Review the introduction briefly, but do not change the writing style nor the messaging;
these are owned by PMMs, so leave it to them to avoid unnecessary back-and-forths.
Make sure feature descriptions make sense, anchors work fine, all internal links have
the relative path.

{{% alert title="Note" color="info" %}}
The introduction or other parts of the release post written may include links to external
blog posts. These links may be broken until the Wednesday, day before release, but should still
be flagged by the TW Lead
during the structural check so the Release Post Manager doesn't miss coordinating
with authors of these external blogs to ensure they're live before the release post blog goes
live
on the release date.
{{% /alert %}}

The Release Post is considered a special blog post instance, so should adhere to the Marketing
editorial team's
style guide.

Making changes

Until 8:00 am Pacific Time on the Monday of release week, the TW Lead should be able to make
changes
directly to the release post. After that time, anyone who wants to include a
change in the upcoming release may need to submit it in a separate MR, with a
target of the release-X-Y branch. For more information, see
Develop on a feature branch.

Frontmatter

In its frontmatter:

- Look for each entry as shown on the code block below.
- Remove any remaining HTML comments and unused blocks to clean up the file.
- Check the title length. The title should be short and deliver an easy-to-understand message
Ensure the title fits nicely with the blog post's title graphic. A general guideline for title
length is about 60 to 70 characters.

yaml

Layout:

The last two entries of the post's frontmatter give the option for a
different layout. If you want to use a dark cover image, you'll need
to uncomment headerlayoutdark: true.

If you want only the release number to be dark, uncomment

releasenumberdark: true.

These two variables work independently; you can assign either of them
or both of them to the same post.

Versioned documentation release

When a new GitLab version is released every month, the Technical Writer who completed the release post structural check for the previous milestone sets up the release of the published documentation for that version.

For instructions, see the GitLab docs monthly release process.

TW Reviewers

{{% alert title="Note" color="info" %}}
TW reviewers should not be confused with the TW lead.
{{% /alert %}}

Each person in the Technical Writing team is responsible for the review
of each individual release post item and deprecation item that falls under their
respective stage/group.

When the PM creates a release post item merge request, or creates a deprecation announcement, they should assign it to the TW
of their group for review (required). The process for TW reviews is described in the:

- Release post item template
- Deprecation MR template

Update the deprecations doc

The deprecations doc is generated with .yml files in gitlab/data/deprecations.

The html pages are not generated automatically. The TW assigned as the reviewer of the deprecation item must run a Rake task to compile the documents. They can also run a separate task to check that the docs are up to date.

While the author of the deprecations MR is responsible for creating the content, they are not responsible for updating the doc.

Updating the docs:

1. From the command line, navigate to your local clone of the gitlab-org/gitlab project, and check out the MR's branch.
1. Compile the deprecation documentation.
1. Commit the updated doc and push the changes.
1. Set the MR to merge when the pipeline succeeds (or merge if the pipeline is already complete).

Deprecation MRs must be merged by the Thursday, 1 week before release. If merged later, they might miss the code cutoff and won't be included in the self-managed release's docs.

If an entry needs to be edited, the update process is similar.

If you run into problems running the Rake task, check the troubleshooting steps.

Product Design Reviewers

{{% alert title="Note" color="info" %}}
Product Designers DRIs review the individual release post items according to the stage/group each designer is assigned to.
{{% /alert %}}

Each PM is responsible for pinging their Product Design counterpart when they need a review on the content or visuals within a release post.

Product Designers should collaborate on release post items and review:

- JTBD: Ensure that the messaging encapsulates how the item supports a user's Job to be Done.
- MVC messaging: Articulate any design vision or future iterations if applicable. This is especially important when considering items that are under construction, or contribute toward a Category's maturity.
- Artifacts: Validate that UI elements (screenshots, GIFs) included in the post are up to date and reflect all design changes. Ensure that no mocks are used.

Engineering Managers

The responsibilities of the Engineering Manager are documented in the Engineering Handbook.

Technical Advisors

Each month, the Release Post Manager may need help with technical hurdles during the release post process. In order to provide the release post, which is a time-sensitive and highly visible asset for customers and users, with adequate technical advisement and support, we are piloting a partnership with the GitLab development team to leverage the Dev Escalation process via the Slack dev-escalation channel as an extension. This ensures that at all times, if something breaks that the release post team can not resolve themselves, they have access to technical experts for resolution. It is recommended that technical advisors review the documented technical aspects of the release post for reference, and the escalation process.

Please note that unlike other monthly volunteers of the release post, the technical advisor is not expected to follow the release post process at all times. The Release Post Manager will reach out to the technical advisor on call via Slack in the dev-escalation channel and then cross-post to the release-post channel for transparency that issues are being worked on. It is then expected that the technical advisor will respond to the Release Post Manager or release post DRI as soon as possible, including evenings/weekends, as the release post asks are often time sensitive, especially between the Monday of release week and the release date of the month. The technical advisor is responsible for determining if further dev escalation should

proceed.

The good news is that the release post technical hurdles are often reasonably easy to troubleshoot for technical experts, which is why we're excited about this partnership!

Below are the types of problems the Release Post Managers may need help with.

- Triaging various automations and technical aspects of the release post
- Triaging pipeline errors and suggest changes or provide a fix to related merge requests
- Resolving merge conflicts with the release post
- Identifying when to engage with other technical teams to resolve upstream problems

Getting help during the Release Post Assembly

Release Post Manager

Should you exhaust your ability to resolve your blocker quickly mention the Technical Advisor in dev-escalation channel and cross-post in release-post channel to ask for help, and make others aware that there may be a delay in assembly.
Describe your blocker in detail, screenshots, videos, etc. can assist in diagnosing the problem. Indicate whether your problem is urgent or not. If you indicate it is urgent, provide a clear date/time by which you need a response or resolution.

Technical Advisor

What we have seen with previous challenges during the Release Post Assembly stage is some difficulty is encountered by the Release Post Manager because of a problem with their local development environment (Ruby setup, permissions, gems, etc.) or git conflicts. You should be familiar with git, Ruby, and the command line. There are a few resources that you can use to diagnose and resolve the issue at hand:

- Review the output of the assembly script including git status
- Consider running ./bin/doctor and review the output
- Reference the list of previous problems

Following your best judgement with the resolution of the incident, record the diagnosis and the steps taken to resolve so that we can improve the release post process and our preparedness. Deposit this info in a new issue or as part of the current release post retrospective.

Automation

We have introduced scheduled pipeline jobs that you should familiarize yourself with:

- A task will run on the Monday, 3 weeks before relea of the month that creates the monthly release post, MRs, and Issues to kickoff the Release Post (pipeline configuration; rake task)
- At <time datetime="16:00">4 pm UTC (11 am ET / 8 am PT)</time>, a task will run that performs content assembly (scheduled pipeline; rake task)

Getting help during the Release Post Deployment

Release Post Manager

Should you exhaust your ability to resolve your blocker quickly mention the Technical Advisor in dev-escalation channel and cross-post in release-post channel to ask for help, and make others aware that there may be a delay in release post deployment.
Describe your blocker in detail, screenshots, videos, etc. can assist in diagnosing the problem. Indicate whether your problem is urgent or not. If you indicate it is urgent, provide a clear date/time by which you need a response or resolution.

Technical Advisor

The Release Post Deployment is a critical and time-sensitive operation. Please respond thoughtfully and quickly.

Following your best judgement with the following:

- For minor incidents that can be recovered from your intervention alone or in concert with the Release Post Manager, do so while recording your diagnosis and the steps taken to resolve the incident so that we can improve the process and our preparedness. Deposit this info in a new issue or as part of the current release post retrospective.
- For major incidents that require immediate assistance from an SRE, developer on call, or other team members with increased access rights, create an issue and follow the dev escalation procedure. Record the diagnosis and the steps taken to resolve so that we can improve the process and our preparedness. Deposit this info in a new issue or as part of the current release post retrospective.

Incident Response

Release post content assembly on the Monday of release week and release post deployment on the release date are time sensitive with multiple dependencies across various departments. GitLab team members often voluntarily go out of their way to assist with blockers found during these two time-sensitive procedures, but it can be confusing as to who is doing what to resolve an active blocking incident. Some procedural detail to our response efforts is shown below.

Response and Resolution SLOs

Due to the time-sensitive nature of both key Release Post actions, assembly and deployment, the initial response time must be very quick, within 15 minutes. Incident resolution should also be as quick, within 60 minutes or less, if possible.

The Role of the Technical Advisor

The introduction of the technical advisor role is meant to be a coordinating role responding to blockers that occur along the way. They may work alone or in tandem with other volunteers to resolve the blocker as they see fit. They are also responsible for clearing the blocker, assembly of others, delegating response tasks including engaging in dev escalation.

Ownership, Positive Control, and Intent

There should only be one owner of an incident at any given time. There must be clear

understanding of who has control of actions to investigate and remedy the incident. Use positive exchange of control, that is pass control to another person who will now be in charge. The extreme example is from aviation where pilots exchange control in a manner like the following where you might hear "your airplane" to pass control followed by "my airplane" from the second pilot to accept control followed by the acknowledgement and release of control from the initiating pilot with "your airplane." This avoids multiple people working at cross purposes from each other. Pilots operating an airplane is an extreme example, but it shows how to use clear language in your efforts to resolve the incident as to who is doing what. Only one person should be have control at a time. Similarly, the person taking action should declare their intent, "I'm going to merge master into the 13.8 release post branch and resolve any conflicts."

Timeline

1. Release Post Manager is blocked. Their initial attempts to get unblocked fails.
2. Release Post Manager joins dev-escalation; mentions the Technical Advisor for this release detailing the nature of the blocker and its severity.
3. Technical Advisor acknowledges that they have seen the message and responds.
4. Technical Advisor creates a dedicated pub